

TDS200 · Kryssplattform

Session 2

Components, JSX & Expo Router Navigation

TypeScript · Tab · Stack · Drawer

Cross-Platform Mobile Development · Lecturer: Yuan Lin

What you'll learn today

- Build screens from core and custom components
- Understand JSX, props, and typing props with TypeScript
- Expo Router Navigation
 - Stack
 - Tab
 - Drawer



Components are LEGO

Small typed pieces compose into screens — exactly like React on the web (DS3103).



Project step

Today you build your app using expo router-based navigation – stack, tab, and drawer navigation.

How a 2-hour session runs

0:00-0:05	Recap & objectives Where we are in the project; today's goals
0:05-0:45	Components, JSX & props, Navigator (theory + demo) Core components, typing props
0:45-1:00	Break —
1:00-1:40	Expo Router Navigation (theory + demo) Build the profile card together
1:40-1:45	Assignment brief & Q&A This week's deliverable, then questions

What is a component?

- A component is a reusable building block of a user interface.
- Benefits: Reusability; Maintainability; Consistency; Easier testing, etc.
- Core components vs. Custom components
 - Core components are provided by React Native: View, Text, Image, TextInput, Pressable, ScrollView, FlatList, etc
 - Custom components are created by developers. The goal is to create reusable custom components.



Components are LEGO

Small typed pieces compose into screens — exactly like React on the web (DS3103).



Project step

Today you build your app using expo router-based navigation – stack, tab, and drawer navigation.

Core components you'll use daily



View

Layout container (like a div)



Text

All text must live in a Text



Image

Local or remote images



Pressable

Touchable / button behavior



ScrollView

Scrollable content area



TextInput

User text entry

A typed, reusable component

components/Post.tsx

- Function components return JSX.
- Props are the component's inputs — type them with an interface.
- TypeScript catches wrong/missing props at compile time.
- In this example, we extract a typed, reusable component called Post.
- The Post component renders an individual post card with a clickable link to the Details page.

```
import { postData } from "@utils/postData";

// A reusable component receives data through props.
// This component knows how to render one post — the parent decides which post to give it.
type PostProps = {
  postData: postData;
};

export default function Post({ postData }: PostProps) {
  return (
    <View style={styles.card}>
      { /* Link navigates to the detail screen, passing the post id as a route parameter */ }
      <Link
        href={{
          pathname: "/home/postDetails/[id]",
          params: { id: postData.id },
        }}
      >
        <Text style={styles.title}>{postData.title}</Text>
        <Text style={styles.description}>{postData.description}</Text>
      </Link>

      <View style={styles.footer}>
        <Text style={styles.hashtags}>{postData.hashtags}</Text>
        <Text style={styles.author}>{postData.author}</Text>
      </View>
    </View>
  );
}
```

A typed, reusable component

index.tsx

- Index.tsx (the home page) renders the FlatList and uses the Post component for each item.

```
import { getAllPosts } from '@api/data/repositories';
import Post from '@components/Post';
import Spacer from '@components/Spacer';

// getAllPosts() returns a static array – no useState needed here.
// In Session 5 we will replace this with a real database call.
export default function HomeScreen() {
  const posts = getAllPosts();

  return (
    <View style={styles.container}>
      <FlatList
        data={posts}
        keyExtractor={({item}) => item.id}
        contentContainerStyle={styles.list}
        ListHeaderComponent={() => <Spacer height={8} />}
        ListFooterComponent={() => <Spacer height={40} />}
        ItemSeparatorComponent={() => <Spacer height={12} />}
        renderItem={({item}) => <Post postData={item} />}
      />
    </View>
  );
}
```

JSX, props & typing props

- We have learned TypeScript in DS3103.
- TypeScript defines and validate prop types.
 - Benefits: compile-time error checking; better IDE support; self-documenting code
 - Some props may be optional, marked by ?

● ● ● Greeting.tsx

```
type Props = {  
  name: string;  
  age: number;  
};  
  
function Greeting({ name, age }: Props) {  
  return <Text>Hi {name}, age {age}</Text>;  
}
```

KEY POINTS

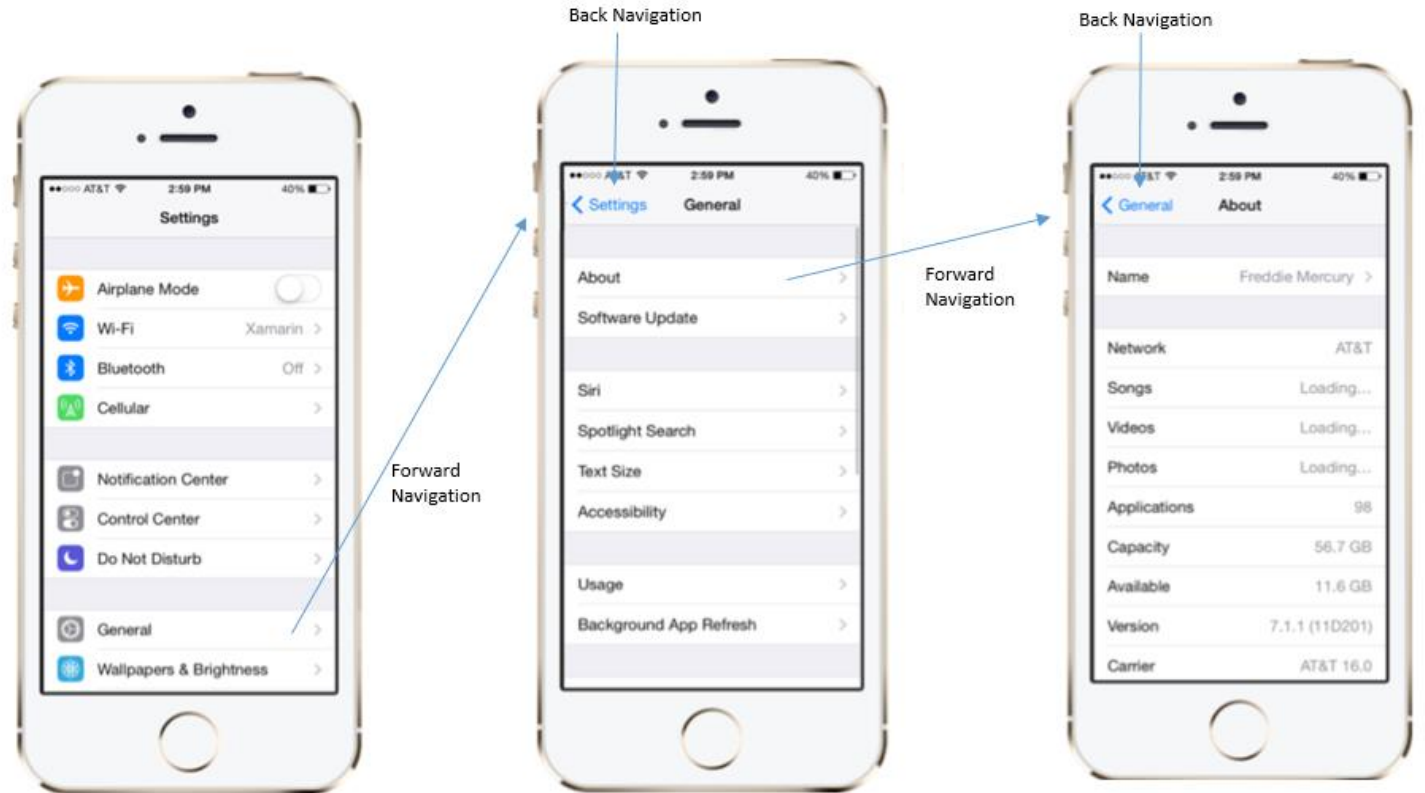
- UI is the value a function returns
- props pass data into the component
- type Props = { ... } types those props

StyleSheet

- StyleSheet is React Native built-in styling system.
 - It defines how components look and are arranged on the screen.
 - Similar to CSS; but written as JavaScript objects.
 - Helps separate presentation from application logic.
 - Makes styles reusable and easier to maintain.
 - StyleSheet definitions is usually located in the end of a React Native file.

Navigation

- Most software allows users to *navigate* in the application
 - A link on a web page takes the user to a new web page.
 - A button in a program opens up a new window
 - Pressing an item in a list changes the current mobile app view / screen / page / activity.
- Navigation systems can be categorized in two main categories:
 - **Stack navigation** - When pressing “back” takes you back to the previous screen.
 - **Free navigation** - When you can navigation from any screen to any other screen.



Expo Router-based Navigation

- Navigation options
 - **Tab navigation:** free navigation, tab buttons at top or bottom of screen
 - **Drawer navigation:** free navigation, navigation menu slides in from a side
 - **Stack navigation:** back / forward
- Navigation in Expo Router is File-based navigation.
 - File-based navigation means that the navigation structure is determined by the files and folders inside the app/ directory.
 - Each screen is represented by a file, and the folder hierarchy automatically defines the available routes.
 - Instead of configuring routes, developers create screens by adding files to the project structure.



Stack

Push/pop screens; a Back button returns to the previous one. Detail pages.



Tabs



Drawer

Package Dependencies

- **Ecosystem requirement:** Session 2 uses a specific set of Expo Router and React Native dependencies (legacy dependencies) to ensure compatibility with **Drawer navigation**. Using newer versions may introduce breaking changes or runtime issues.
- **Below are the list of legacy dependencies:**
 - expo ~53.0.22, expo-router ~5.1.5
 - react-native 0.79.6, reanimated ~3.17.4, gesture-handler ~2.24.0
 - @expo/vector-icons ^14.1.0
- We will switch back to latest dependencies from Session 3.
- If you have expo-f0 fetch failed error, try:

npx expo-go download ios 53

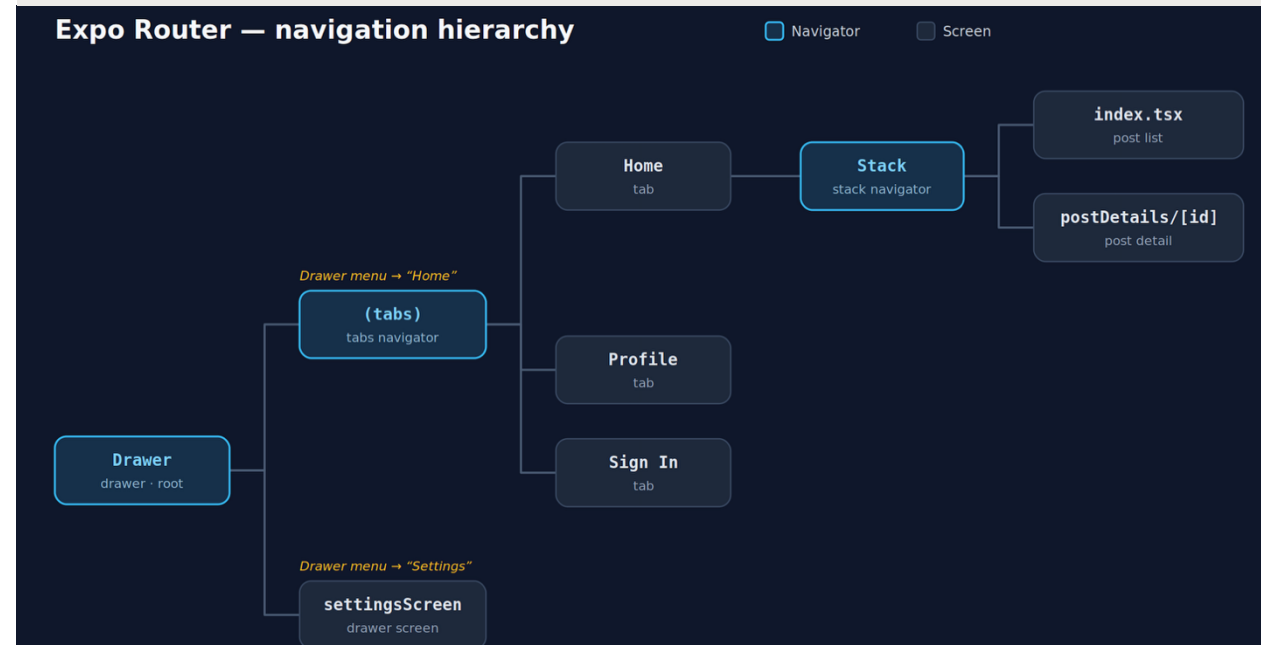
npx expo start --clear

```
npm install expo@~53.0.22 \
expo-router@~5.1.5 \
react-native@0.79.6 \
react-native-reanimated@~3.17.4 \
react-native-gesture-handler@~2.24.0 \
@expo/vector-icons@^14.1.0
```

```
"dependencies": {
  "@expo/vector-icons": "^14.1.0",
  "@react-navigation/drawer": "^7.0.0",
  "expo": "~53.0.22",
  "expo-asset": "^56.0.17",
  "expo-font": "~13.0.3",
  "expo-router": "~5.1.5",
  "expo-splash-screen": "~0.29.22",
  "expo-status-bar": "~2.2.3",
  "react": "19.0.0",
  "react-dom": "19.0.0",
  "react-native": "0.79.6",
  "react-native-gesture-handler": "~2.24.0",
  "react-native-reanimated": "~3.17.4",
  "react-native-safe-area-context": "5.0.0",
  "react-native-screens": "~4.3.0",
  "react-native-web": "~0.20.0"
}.
```

Expo Router based Navigation

- The navigation hierarchy is as shown.



Drawer at the Root

- The app/_layout.tsx defines the Drawer navigator, which is the root navigator.
- The Drawer manages the application's top-level navigation.
 - Home - /(tabs)
 - Settings - /settingsScreen
- App/index.tsx works as the entry point of the app and represents the default root URL "/". But it does not contain a real page, it just redirects to /home.
- When you click Home at Drawer menu, it open (tabs)/_layout.tsx. And creates the Tabs navigator. It does not directly display the default Tab.



```
export default function Index() {
  return <Redirect href="/home" />;
}
```

```
export default function RootLayout() {
  return [
    <GestureHandlerRootView style={{ flex: 1 }}>
      <Drawer>
        <Drawer.Screen
          name="index"
          options={{ drawerItemStyle: { display: "none" } }}
        />
        <Drawer.Screen
          name="(tabs)"
          options={{
            headerShown: false,
            drawerLabel: "Home",
          }}
        />
        <Drawer.Screen
          name="settingsScreen"
          options={{
            title: "Settings",
            drawerLabel: "Settings",
          }}
        />
      </Drawer>
    </GestureHandlerRootView>
  ];
}
```

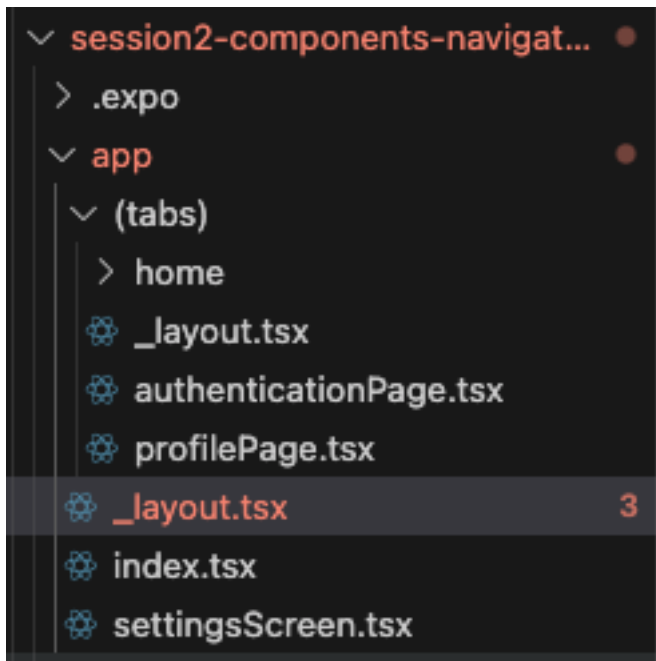
Drawer at the Root

- drawerItemStyle: { display: "none" } means the drawer should not display an item for it. Index exists for routing purpose.
- headerShown:false prevents Drawer navigator from adding the header

```
export default function RootLayout() {
  return (
    <GestureHandlerRootView style={{ flex: 1 }}>
      <Drawer>
        <Drawer.Screen
          name="index"
          options={{ drawerItemStyle: { display: "none" } }}
        />
        <Drawer.Screen
          name="(tabs)"
          options={{
            headerShown: false,
            drawerLabel: "Home",
          }}
        />
        <Drawer.Screen
          name="settingsScreen"
          options={{
            title: "Settings",
            drawerLabel: "Settings",
          }}
        />
      </Drawer>
    </GestureHandlerRootView>
  );
}
```

Which file creates the URL /home?

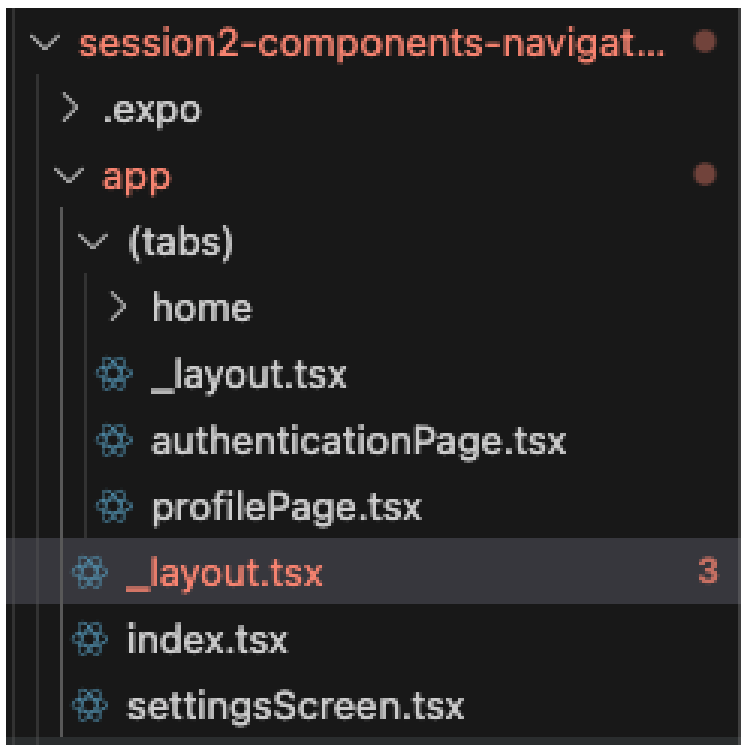
- Expo router looks for a file that created the URL /home.
- **App/(tabs)/home/index.tsx** creates the url /home. Why?
 - (tabs) means a Route group that is not included in URL.
 - Home becomes the /home URL segment.
 - index.tsx represents the default page of its folder and adds no URL.



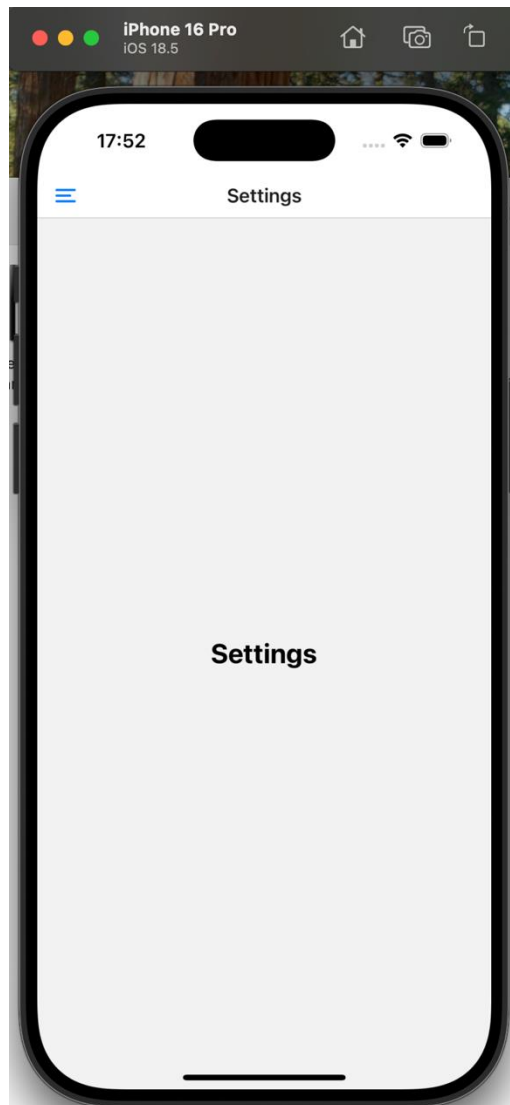
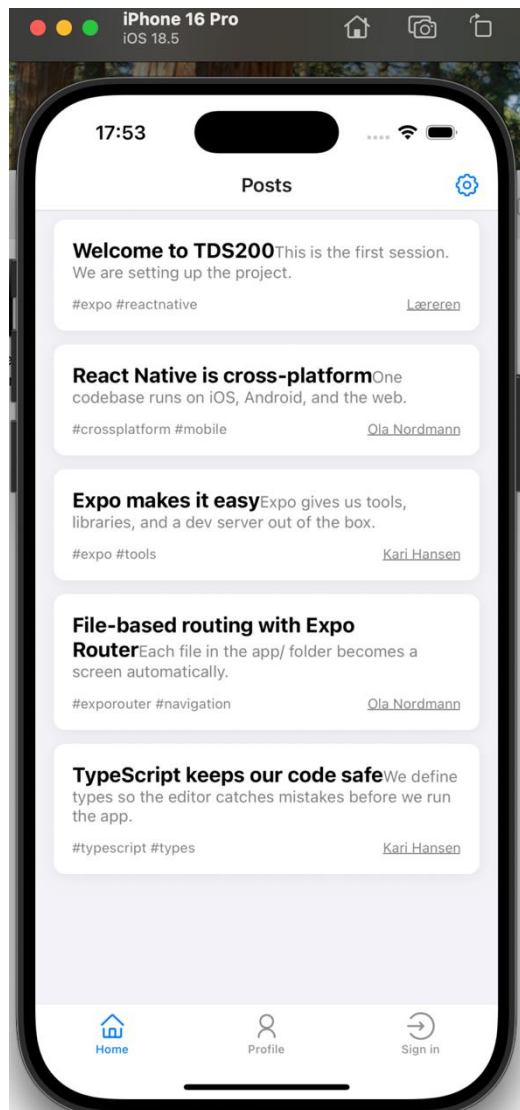
```
export default function TabsLayout() {
  return (
    <Tabs>
      <Tabs.Screen
        name="home"
        options={{
          title: "Home",
          headerShown: false,
          tabBarIcon: ({ color }) => (
            <AntDesign name="home" size={24} color={color} />
          ),
        }}
      />
      <Tabs.Screen
        name="profilePage"
        options={{
          title: "Profile",
          tabBarIcon: ({ color }) => (
            <AntDesign name="user" size={24} color={color} />
          ),
        }}
      />
      <Tabs.Screen
        name="authenticationPage"
        options={{
          title: "Sign in",
          tabBarIcon: ({ color }) => (
            <AntDesign name="login" size={24} color={color} />
          ),
        }}
      />
    </Tabs>
  );
}
```

Which file creates the URL /home?

- What if I also have a home page under the root app folder?
- If you have `app/home.tsx`, it creates URL `/home`. While `app/(tabs)/home/index.tsx` also creates `/home` URL. This creates a route conflict. Expo router can not reliably determine which `/home` screen you point to. Avoid having both.

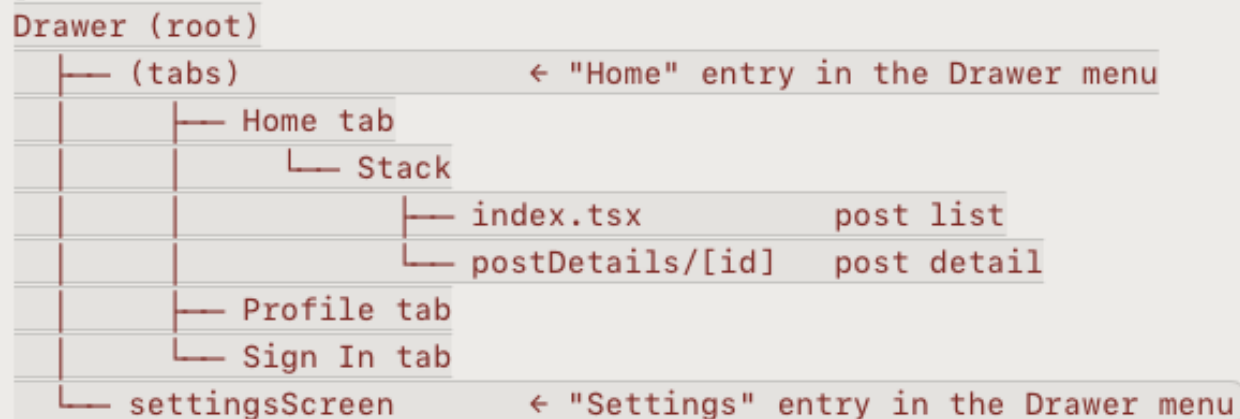


Drawer navigator



Tab navigator

- When you click Home in the Drawer, it loads (tabs). (tabs) is a route group and `app/(tabs)/_layout.tsx` defines the Tab navigator.
- Inside `app/(tabs)/_layout.tsx`, the Home tab points to `app/(tabs)/home` folder.
- The code on the right is `app/(tabs)/_layout.tsx`
- Note that **name** attribute is the url path, that connects a navigator screen to a file or folder in app directory.



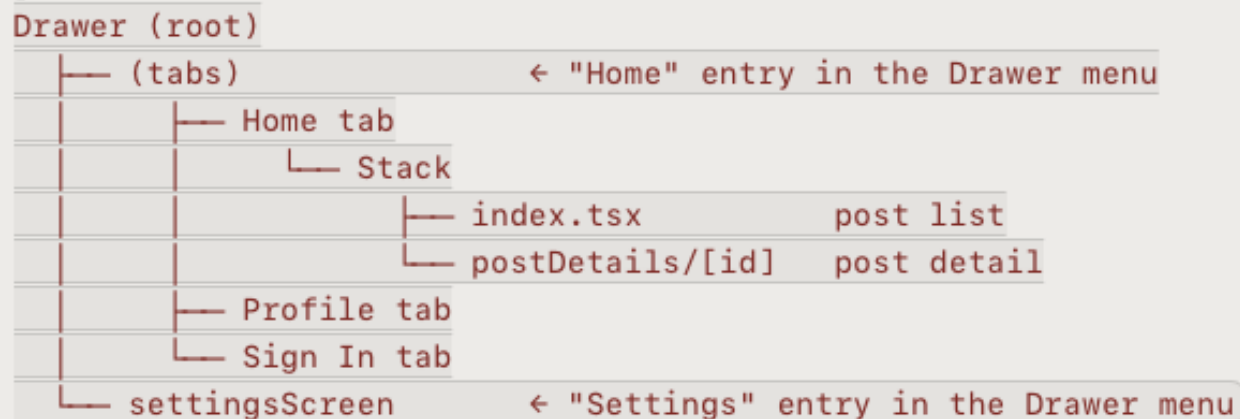
```

export default function TabsLayout() {
  return (
    <Tabs>
      <Tabs.Screen
        name="home"
        options={{
          title: "Home",
          headerShown: false,
          tabBarIcon: ({ color }) => (
            <AntDesign name="home" size={24} color={color} />
          ),
        }}
      />
      <Tabs.Screen
        name="profilePage"
        options={{
          title: "Profile",
          tabBarIcon: ({ color }) => (
            <AntDesign name="user" size={24} color={color} />
          ),
        }}
      />
      <Tabs.Screen
        name="authenticationPage"
        options={{
          title: "Sign in",
          tabBarIcon: ({ color }) => (
            <AntDesign name="login" size={24} color={color} />
          ),
        }}
      />
    </Tabs>
  );
}

```

Tab navigator

- If you have defined `initialRouteName`, such as `<Tabs initialRouteName="home">`, it selects Home tab when you loads the Tab Navigator.
- Without defining `initialRouteName`, it finds the first available Tab.
- Because your root `index.tsx` explicitly redirects to `/home`, so home is the default Tab when you initiate the app.

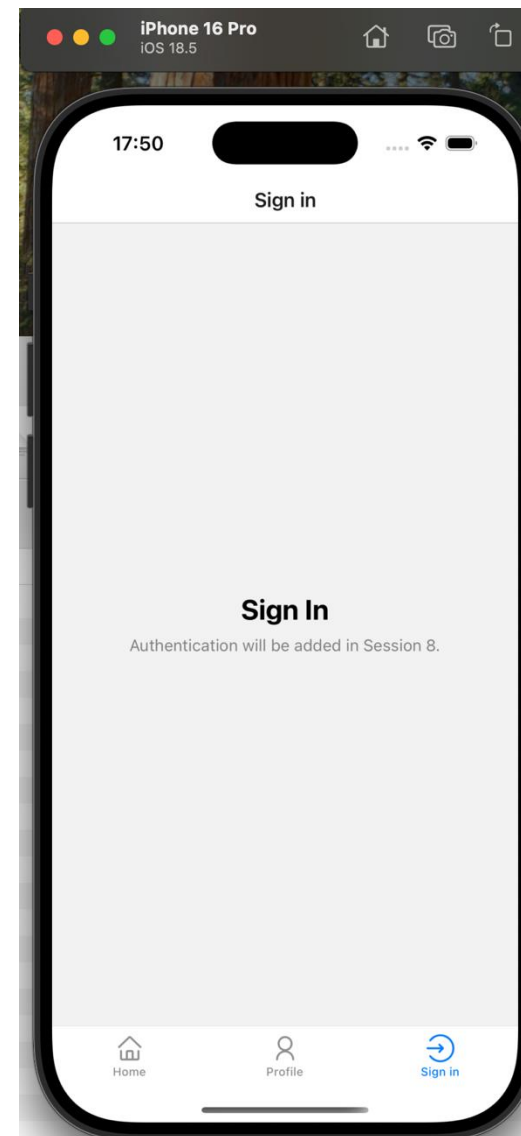
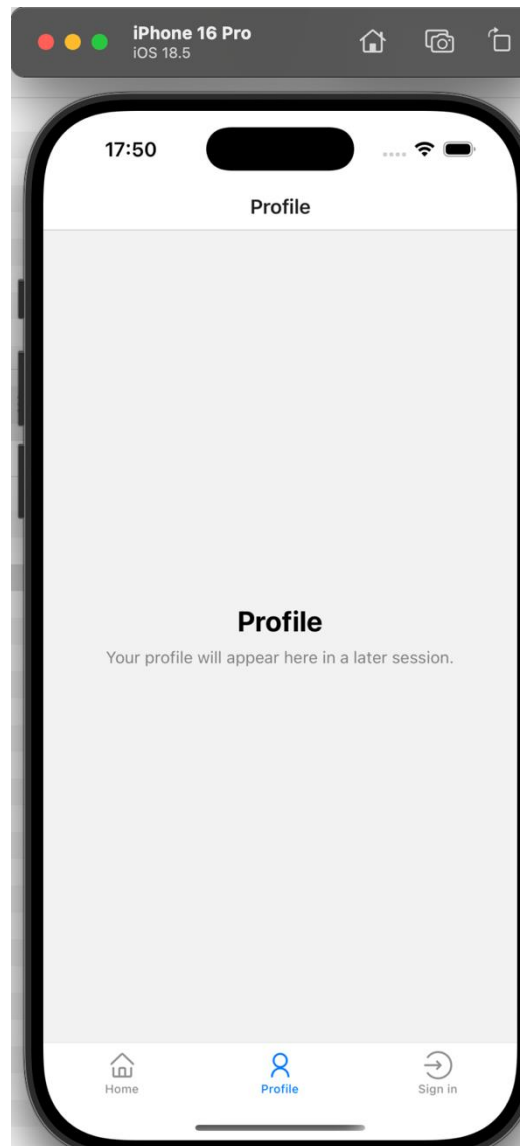
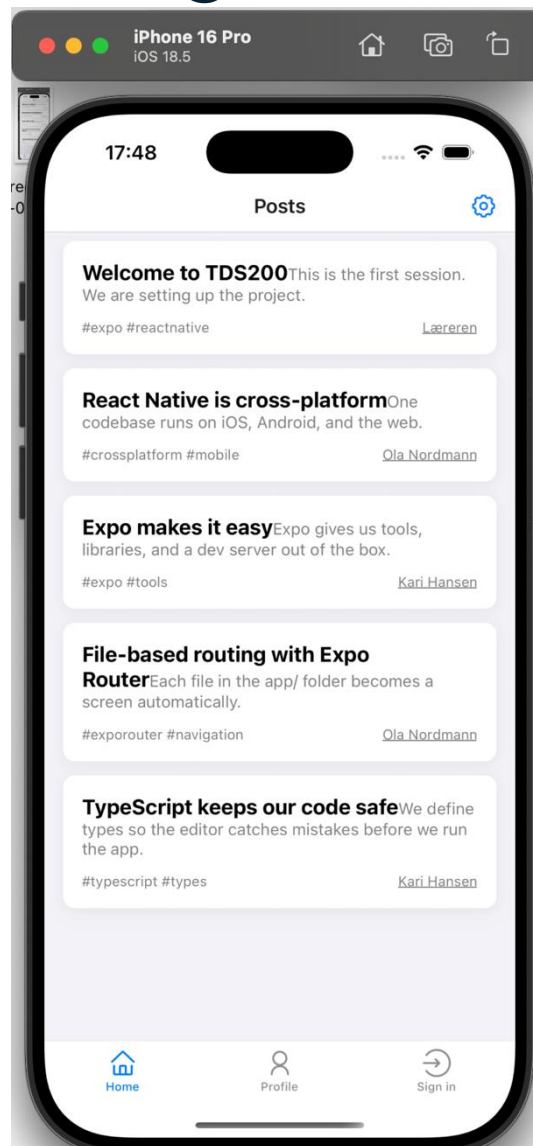


```

export default function TabsLayout() {
  return (
    <Tabs>
      <Tabs.Screen
        name="home"
        options={{
          title: "Home",
          headerShown: false,
          tabBarIcon: ({ color }) => (
            <AntDesign name="home" size={24} color={color} />
          ),
        }}
      />
      <Tabs.Screen
        name="profilePage"
        options={{
          title: "Profile",
          tabBarIcon: ({ color }) => (
            <AntDesign name="user" size={24} color={color} />
          ),
        }}
      />
      <Tabs.Screen
        name="authenticationPage"
        options={{
          title: "Sign in",
          tabBarIcon: ({ color }) => (
            <AntDesign name="login" size={24} color={color} />
          ),
        }}
      />
    </Tabs>
  );
}

```

Tab navigator



Stack Navigator

- After Expo navigates to the home folder, it then looks for `home/_layout.tsx`. That layout defines a Stack navigator.
- A Stack displays screens on top of one another. When you open a post, the details screen is pushed onto the Stack. Going back removes it and reveals the posts screen again.
- `Index.tsx` represents the default route of its folder. When Expo router navigates to `/home`, it loads `/home/index.tsx`.

Home tab -> home/ -> home/_layout.tsx -> home/index.tsx



```

export default function HomeLayout() {
  const router = useRouter();

  return (
    <Stack>
      <Stack.Screen
        name="index"
        options={{
          title: "Posts",
          headerRight: () => (
            <TouchableOpacity onPress={() => router.push("/settingsScreen")}>
              <AntDesign name="setting" size={22} color="#007AFF" />
            </TouchableOpacity>
          ),
        }}
      />
      <Stack.Screen
        name="postDetails/[id]"
        options={{
          title: "Post Details",
          headerLeft: () => (
            <TouchableOpacity onPress={() => router.back()}>
              <AntDesign name="arrowleft" size={22} color="#007AFF" />
            </TouchableOpacity>
          ),
        }}
      />
    </Stack>
  );
}

```

Read params on the detail screen

postDetails/[id].tsx

- Link to a post with its id in the path.
- `useLocalSearchParams()` reads the `[id]` value.
- Same screen, different data per id.
- Where is the id from? The value comes from the route used during navigation, since you defined the pathname in Link in Post component, and `id=postData.id`

```
// useLocalSearchParams() reads the [id] segment from the URL.
// When we navigate to /home/postDetails/p1, id will be "p1".
export default function PostDetailsScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const post = getPostById(id);

  if (!post) {
    return (
      <View style={styles.container}>
        <Text style={styles.error}>Post not found.</Text>
      </View>
    );
  }

  return (
    <View style={styles.container}>
      <Text style={styles.title}>{post.title}</Text>
      <Text style={styles.author}>By {post.author}</Text>
      <Text style={styles.description}>{post.description}</Text>
      <Text style={styles.hashtags}>{post.hashtags}</Text>
    </View>
  );
}
```

```
<Link
  href={{
    pathname: "/home/postDetails/[id]",
    params: { id: postData.id },
  }}
  asChild
/>
```

Link

- Link navigates to the detail screen, passing the post id as a route parameter.
- Inside Link, params supplies id to pathname, replacing [id] with a real value.
- The file path is determined by expo file-based routing system, which points to app/(tabs)/home/postDetails/[id].tsx in the file system.
- From web version, you will see the url is home/postDetails/[id], provided by pathname.

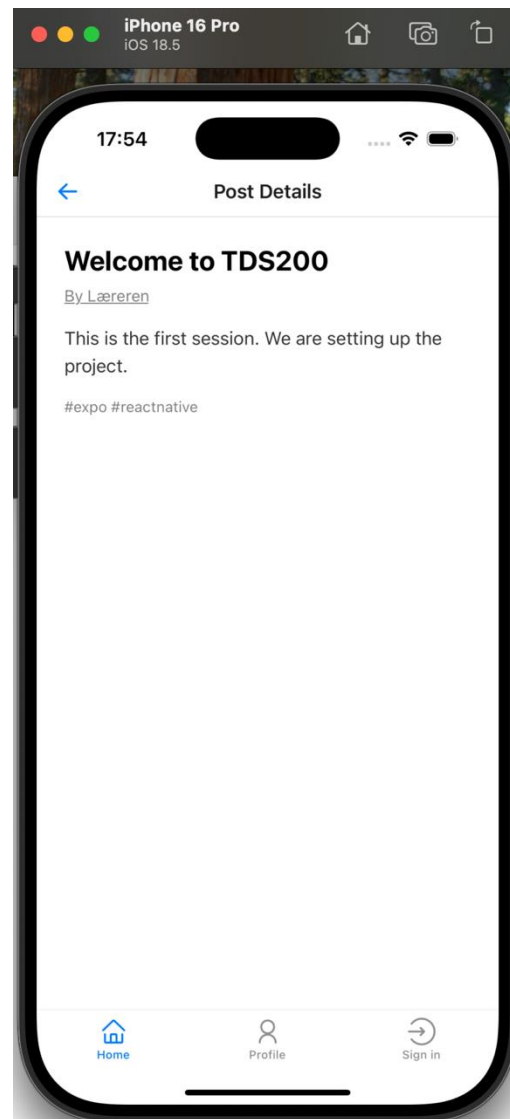
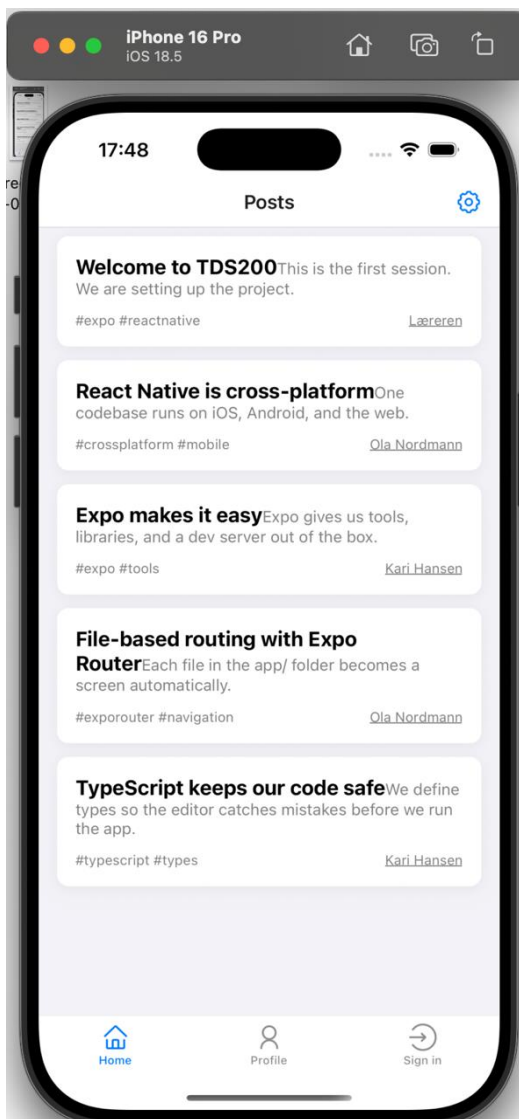
Post.tsx

```
// A reusable component receives data through props.
// This component knows how to render one post – the parent decides which post to give it.
type PostProps = {
  postData: PostData;
};

export default function Post({ postData }: PostProps) {
  return (
    <View style={styles.card}>
      { /* Link navigates to the detail screen, passing the post id as a route parameter */ }
      <Link
        href={{
          pathname: "/home/postDetails/[id]",
          params: { id: postData.id },
        }}
        asChild
      >
        <TouchableOpacity>
          <Text style={styles.title}>{postData.title}</Text>
          <Text style={styles.description}>{postData.description}</Text>
        </TouchableOpacity>
      </Link>

      <View style={styles.footer}>
        <Text style={styles.hashtags}>{postData.hashtags}</Text>
        <Text style={styles.author}>{postData.author}</Text>
      </View>
    </View>
  );
}
```

Stack navigator



A Wrapping-up

- The current URL/route decides which file is displayed. `_layout.tsx` does not create the content.
- Who is responsible for what?
 - `Link/router.push()` changes the current route.
 - Expo router matches the route to the correct file.
 - File: returns the page content.
 - `_layout.tsx` chooses the navigator and the page layout/organization.
 - Screen configures the matched screen.

Expo Router-based Navigation - Tips

- When you use Expo router, the app/ folder is mandatory. It is where the application's navigation structure is defined.
- Each file inside app/ automatically maps to a route. Example:
 - app/index.tsx -> /
 - app/profile.tsx -> /profile
- Use square brackets [] to define dynamic routes, i.e.,
 - app/postDetails/[id].tsx -> /postDetails/id
- Use parentheses () to create route groups that do not appear in the URL, i.e.,
 - app/(tabs)/home.tsx -> /home
- Use <Link> component or router.push() to navigate between screens.
- _layout.tsx files define navigators and layouts for a folder and its child routes.
- index.tsx is a special file in Expo Router.
 - It represents the default route of a folder. It is equivalent to the home page of that folder.
 - When users navigate to the folder path, Expo Router automatically loads index.tsx.
 - If you do not have index.tsx in a folder, unmatched route might appear if you try to access the root url of that folder.

Keep the development loop tight

- **Run the project and leave it running.**
 - `npx expo start`, then press `i` / `a`, or scan in Expo Go.
 - Fast Refresh re-renders on every save — no manual reload.
- What is the difference between `npx expo start` and `npx expo run:ios`?
 - `npx expo start` starts the expo development server called Metro server. It does not build the native iOS app. When you open an iOS simulator from the terminal, it is through Expo Go.
 - `npx expo run:ios` builds and installs native iOS app, then starts Metro. It creates `ios` folder. When you need native modules, custom native configuration or your own development build for deployment, you need `run:ios`.

terminal

```
npx expo start

# i  → iOS Simulator
# a  → Android Emulator
# r  → reload    m → dev menu
```

TIP

Layout looks different per platform — always check both.



Tag Component

YOUR TASKS

- Task 1: The post detail screen (`app/(tabs)/home/postDetails/[id].tsx`) currently displays hashtags as a single plain-text string (e.g., `#expo #reactnative`). They come from `PostData.hashtags` field. Your job is to build a reusable `Tag` component and use it to display each hashtag as an individual styled chip on the detail screen. You will also use the existing `Spacer` component to improve the spacing.
- Task 2: remove the root `Drawer` navigator and simplify the app navigator to use only `Tab` and `Stack` navigators.

DELIVERABLE

A `Tag` component that is reusable.

EST. EFFORT

≈ 3 hours